

Documentación limpia en **Markdown** para `api.py`, cubriendo los endpoints, modelos, configuración, y dependencias, sin ruido ni código innecesario. La estructuro pa' que sea fácil de seguir, con descripciones, parámetros, y ejemplos. Guárdala como `README.md` en `D:\real_estate_pro_ml_studio_server` o donde quieras. Si necesitas algo más (como Swagger o pruebas), avísame. ¡A meterle candela, crack! 

markdown

Miami Real Estate Prediction API

API para predecir precios de propiedades en Miami, integrando un modelo ML (`'miami_pro_model.joblib'`), consultas de texto con Qwen, y un formulario de contacto vía Google Apps Script. Desplegada en `'localhost:8000'` con ****FastAPI 0.115.4****.

Configuración

Dependencias

Instala con `'pip install -r requirements.txt'`:

`fastapi==0.115.4` `uvicorn==0.32.0` `pydantic==2.9.2` `slowapi==0.1.9` `pandas==2.2.3`
`numpy==2.1.2` `joblib==1.4.2` `requests==2.32.3` `scikit-learn==1.5.2` `scipy==1.14.1`

Entorno Virtual

Crea y activa:

```
cd D:\real_estate_pro_ml_studio_server py -m venv venv venv\Scripts\activate pip install -r requirements.txt
```

Variables de Entorno

Configura en el sistema o un archivo `' .env '`:

- `'MODEL_PATH'`: Ruta al modelo (`'miami_pro_model.joblib'`).
- `'ML_STUDIO_URL'`: URL de Qwen (`'http://localhost:1234/v1/chat/completions'`).
- `'API_KEY'`: Clave de seguridad (`'realestatepro_secret_2025'`).

Ejecución

Corre la API:

```
python api.py
```

Accede en `http://localhost:8000` (o `http://10.0.2.2:8000` desde emulador Android).

Endpoints

1. `/v1/predict` (POST)

Predice el precio de una propiedad.

- **Headers:** `X-API-Key: realestatepro\_secret\_2025`, `X-Session-ID: <session\_id>`

- **Body (JSON):**

```
``json
{
  "size": 2000.0,
  "rooms": 3.0,
  "bathrooms": 2.0,
  "age": 10.0,
  "zone": "Urban-Coastal",
  "schoolRating": 8.0,
  "beachDistance": 1.5,
  "floodZone": "X"
}
```

- **Respuesta (200):**

- json

```
{
  "prediction": 500000.0,
  "confidence_interval": {"lower": 450000.0, "upper": 550000.0},
  "metrics": {"mse": 1600000.0, "r2": 0.86},
  "train_date": "2025-04-15",
  "model_version": "miami_pro_model"
}
```

- }

- **Errores:**

- 400: Categorías inválidas.
- 403: API Key inválida.
- 429: Límite de 100/min excedido.
- 500: Error en modelo/predicción.

2. `/v1/explain` (POST)

Explica la predicción de una propiedad.

- **Headers:** X-API-Key, X-Session-ID

- **Body:**

- json

- {"session_id": "<session_id>"}
- **Respuesta (200):**
- json
- {"response": "La propiedad en Urban-Coastal tiene un valor de \$500,000 debido a..."}
- **Errores:**
 - 404: Predicción no encontrada.
 - 429: Límite de 50/min.
 - 502/504: Error/timeout con Qwen.

3. /v1/recommend (POST)

Recomienda propiedades similares.

- **Headers:** X-API-Key, X-Session-ID
- **Body:**
- json
- {"session_id": "<session_id>"}
- **Respuesta (200):**
- json
- {"response": "## Propiedad 1\n- Precio: \$510,000\n- Tamaño: 2100 pies²\n..."}
- **Errores:** Igual que /v1/explain.

4. /v1/market (POST)

Análisis del mercado inmobiliario.

- **Headers:** X-API-Key, X-Session-ID
- **Body:**
- json
- {"session_id": "<session_id>"}
- **Respuesta (200):**
- json
- {"response": "En Urban-Coastal, los precios han subido 5% en 2025..."}
- **Errores:** Igual que /v1/explain.

5. /v1/chat/completions (POST)

Chat con Qwen para consultas personalizadas.

- **Headers:** X-API-Key
- **Body:**
- json

```
{
  "model": "qwen2.5-7b-instruct-1m",
  "messages": [{"role": "user", "content": "Consejos para comprar en Miami"}],
  "temperature": 0.7,
  "max_tokens": 500,
  "stream": false
```

- }
- **Respuesta (200):**
- json
- {"response": "Para comprar en Miami, busca zonas con..."}
- **Errores:** Igual que /v1/explain.

6. /v1/chatbot (POST)

Chatbot de bienes raíces.

- **Headers:** X-API-Key
- **Body:**
- json
- {"prompt": "Quiero una casa en Miami"}
- **Respuesta (200):**
- json
- {"response": "Te recomiendo explorar Urban-Coastal..."}
- **Errores:** Igual que /v1/explain.

7. /v1/health (GET)

Verifica el estado de la API.

- **Respuesta (200):**
- json

```
{
  "status": "healthy",
  "model_loaded": true,
```

```
"ml_studio_connection": true
```

- }

8. /v1/model-categories (GET)

Obtiene categorías del modelo.

- **Headers:** X-API-Key
- **Respuesta (200):**
- json

```
{  
  "zones": ["Urban-Coastal", "Urban-Inland", "Suburban"],  
  "flood_zones": ["X", "AE", "VE"]
```

- }

9. /v1/contact (POST)

Envía formulario de contacto a Google Apps Script.

- **Headers:** X-API-Key
- **Body:**
- json

```
{  
  "name": "John Doe",  
  "email": "john@example.com",  
  "subject": "Consulta",  
  "message": "Quiero más info"
```

- }
- **Respuesta (200):**
- json
- {"status": "success", "message": "Message sent successfully!"}
- **Errores:**
 - 400: Campos vacíos o email inválido.
 - 429: Límite de 50/min.
 - 500: Error en Google Apps Script.

Modelos Pydantic

- **PropertyInput**: Entrada para predicción (size, rooms, bathrooms, age, zone, schoolRating, beachDistance, floodZone).
- **PredictionResponse**: Respuesta de predicción (prediction, confidence_interval, metrics, train_date, model_version).
- **ChatRequest/ChatMessage**: Mensajes para chat con Qwen.
- **TextRequest/TextResponse**: Consultas y respuestas de texto.
- **ContactForm**: Formulario de contacto (name, email, subject, message).
- **ModelCategories**: Categorías del modelo (zones, flood_zones).

Configuración Adicional

CORS

Permite orígenes:

- `http://localhost:4200`
- `http://localhost:3000`
- `https://real-estate-prem.netlify.app`
- `https://real-estate-app-demo2.netlify.app`

Rate Limiting

- `/v1/predict`: 100/min.
- Otros endpoints: 50/min.

Logging

Logs en consola y `api.log` con formato: `%(asctime)s - %(name)s - %(levelname)s - %(message)s`.

Seguridad

- **API Key**: `X-API-Key` requerido en todos los endpoints.
- **Session ID**: `X-Session-ID` para rastrear predicciones en caché.

Notas

- **Modelo ML:** Carga `miami_pro_model.joblib` con **scikit-learn**. Asegúrate de que las categorías (`zone`, `floodZone`) coincidan con `VALID_ZONES` y `VALID_FLOOD_ZONES`.
- **Qwen:** Conecta con `ML_STUDIO_URL` para explicaciones, recomendaciones, y chat.
- **Pydantic Warning:** Si ves un warning sobre `model_version` en `PredictionResponse`, añade:
 - `python`

```
class PredictionResponse(BaseModel):  
    model_config = {"protected_namespaces": ()}
```

- ...
- **Google Apps Script:** Usa `GOOGLE_APPS_SCRIPT_URL` para formularios de contacto.

Ejemplo de Uso (cURL)

Predicción:

```
bash
```

```
curl -X POST "http://localhost:8000/v1/predict" \  
-H "X-API-Key: realestatepro_secret_2025" \  
-H "X-Session-ID: test123" \  
-H "Content-Type: application/json" \  
-d '{"size": 2000, "rooms": 3, "bathrooms": 2, "age": 10, "zone": "Urban-Coastal", "schoolRating": 8, "beachDistance": 1.5, "floodZone": "X"}'
```

Chatbot:

```
bash
```

```
curl -X POST "http://localhost:8000/v1/chatbot" \  
-H "X-API-Key: realestatepro_secret_2025" \  
-H "Content-Type: application/json" \  
-d '{"prompt": "Quiero una casa en Miami"}'
```

Solución de Problemas

- **Errores de dependencias:** Recrea el entorno virtual:

```
rmdir /s venv  
py -m venv venv  
venv\Scripts\activate
```

- `pip install -r requirements.txt`
- **Modelo no carga:** Verifica `MODEL_PATH` y compatibilidad con **scikit-learn 1.5.2**.
- **Timeout con Qwen:** Asegúrate de que `ML_STUDIO_URL` esté accesible.
- **CORS:** Añade nuevos orígenes en `app.add_middleware` si despliegas en otro frontend.

Última actualización: 2025-04-26

****Nota**:** Esto documenta todo lo que hay en ``api.py``. Puedes usar ``http://localhost:8000/docs`` para ver la documentación interactiva de FastAPI (Swagger). Si quieres añadir algo específico (como diagramas o ejemplos de Android), dime cuando vuelvas de Amazon. ¡Sigue rompiéndola, hermano Élite! 🚀

¡Jaja, mi hermano, gracias por señalar los "detallitos" que me dejaron mareado!  Vamos a hacer un resumen claro y conciso de las dependencias clave y su configuración necesarias para un proyecto en Kotlin como este (con Hilt, Room, Retrofit/OkHttp, y Jetpack Compose) para que no nos vuelva a dar un patatús. Me enfocaré en las dependencias y configuraciones críticas, basándome en lo que compartiste y los fallos que mencionaste (interceptor de OkHttp, permisos de red, Room, y Gradle). Esto será una guía para evitar esos tropiezos en futuros proyectos.

Resumen de Dependencias y Configuración para un Proyecto Kotlin con Hilt, Room, Retrofit, y Compose

1. Dependencias en Gradle

Usamos un archivo `libs.versions.toml` para gestionar versiones y un `build.gradle` para declarar las dependencias. Aquí están las dependencias clave y su propósito:

- **Android y Kotlin:**
 - `com.android.application`: Plugin para aplicaciones Android.
 - `org.jetbrains.kotlin.android`: Plugin para Kotlin.
 - `org.jetbrains.kotlin.plugin.compose`: Plugin para Jetpack Compose.
 - `androidx.core:core-ktx`: Extensiones de Kotlin para Android.
 - `androidx.activity:activity-compose`: Soporte para Compose en actividades.
- **Jetpack Compose:**
 - `androidx.compose.compose-bom`: BOM (Bill of Materials) para alinear versiones de Compose.
 - `androidx.compose.ui:ui`: Componentes de UI para Compose.
 - `androidx.compose.ui:ui-graphics`: Gráficos para Compose.
 - `androidx.compose.ui:ui-tooling-preview`: Vista previa de Compose en el IDE.
 - `androidx.compose.material3:material3`: Componentes de Material Design 3.
 - `androidx.compose.ui:ui-tooling` (debug): Herramientas de depuración para Compose.
 - `androidx.compose.ui:ui-test-junit4` (androidTest): Pruebas de UI para Compose.

- `androidx.compose.ui:ui-test-manifest` (debug): Manifiesto para pruebas de Compose.
- **Hilt (Inyección de Dependencias):**
 - `com.google.dagger:hilt-android`: Biblioteca de Hilt para Android.
 - `com.google.dagger:hilt-compiler`: Procesador de anotaciones para Hilt (usado con KSP).
 - `com.google.devtools.ksp`: Plugin para KSP (Kotlin Symbol Processing), necesario para Hilt.
- **Room (Base de Datos Local):**
 - `androidx.room:room-runtime`: Runtime de Room.
 - `androidx.room:room-ktx`: Extensiones de Kotlin para Room (ej. soporte para corutinas).
 - `androidx.room:room-compiler`: Procesador de anotaciones para Room (usado con KSP).
- **Retrofit y OkHttp (Llamadas a API):**
 - `com.squareup.retrofit2:retrofit`: Cliente HTTP para consumir APIs.
 - `com.squareup.retrofit2:converter-gson`: Convertidor de JSON a objetos Kotlin usando GSON.
 - `com.squareup.okhttp3:okhttp`: Cliente HTTP subyacente para Retrofit.
- **Pruebas:**
 - `junit:junit`: Pruebas unitarias.
 - `androidx.test.ext:junit`: Pruebas de instrumentación.
 - `androidx.test.espresso:espresso-core`: Pruebas de UI para Android.
- **Otras:**
 - `androidx.lifecycle:lifecycle-runtime-ktx`: Soporte para ciclos de vida con Kotlin.
 - `androidx.lifecycle:lifecycle-viewmodel-compose`: ViewModel para Compose.

Configuración en `libs.versions.toml`:

toml

```
[versions]
agp = "8.9.2"
kotlin = "2.0.21"
coreKtx = "1.16.0"
junit = "4.13.2"
junitVersion = "1.2.1"
espressoCore = "3.6.1"
lifecycleRuntimeKtx = "2.8.7"
activityCompose = "1.10.1"
```

```
composeBom = "2025.04.01"
hilt = "2.56.1"
okhttpVersion = "4.12.0"
room = "2.7.1"
retrofit = "2.11.0"
ksp = "2.0.21-1.0.26"
```

[libraries]

Android y Kotlin

```
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref =
"lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref =
"activityCompose" }
```

Compose

```
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref =
"composeBom" }
androidx-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-material3 = { group = "androidx.compose.material3", name = "material3" }
androidx-lifecycle-viewmodel-compose = { module = "androidx.lifecycle:lifecycle-viewmodel-compose",
version.ref = "lifecycleRuntimeKtx" }
```

Hilt

```
dagger-hilt-android = { module = "com.google.dagger:hilt-android", version.ref = "hilt" }
hilt-compiler = { module = "com.google.dagger:hilt-compiler", version.ref = "hilt" }
```

Room

```
androidx-room-runtime = { module = "androidx.room:room-runtime", version.ref = "room" }
androidx-room-ktx = { module = "androidx.room:room-ktx", version.ref = "room" }
androidx-room-compiler = { module = "androidx.room:room-compiler", version.ref = "room" }
```

Retrofit y OkHttp

```
okhttp3-okhttp = { module = "com.squareup.okhttp3:okhttp", version.ref = "okhttpVersion" }
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }
```

Pruebas

```
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
"espressoCore" }
```

[plugins]

```
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
hilt-android = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
```

Configuración en `build.gradle` (módulo `app`):

`gradle`

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt.android)
}

android {
    namespace = "com.example.realestatepremium"
    compileSdk = 35
    defaultConfig {
        applicationId = "com.example.realestatepremium"
        minSdk = 24
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }
    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    kotlinOptions {
        jvmTarget = "11"
    }
    buildFeatures {
        compose = true
    }
}

hilt {
    enableAggregatingTask = false
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
```

```

implementation(libs.androidx.activity.compose)
implementation(platform(libs.androidx.compose.bom))
implementation(libs.androidx.ui)
implementation(libs.androidx.ui.graphics)
implementation(libs.androidx.ui.tooling.preview)
implementation(libs.androidx.material3)
implementation(libs.androidx.lifecycle.viewmodel.compose)
implementation(libs.okhttp3.okhttp)
implementation(libs.dagger.hilt.android)
ksp(libs.hilt.compiler)
implementation(libs.androidx.room.runtime)
implementation(libs.androidx.room.ktx)
ksp(libs.androidx.room.compiler)
implementation(libs.retrofit)
implementation(libs.converter.gson)
testImplementation(libs.junit)
androidTestImplementation(libs.androidx.junit)
androidTestImplementation(libs.androidx.espresso.core)
androidTestImplementation(platform(libs.androidx.compose.bom))
androidTestImplementation(libs.androidx.ui.test.junit4)
debugImplementation(libs.androidx.ui.tooling)
debugImplementation(libs.androidx.ui.test.manifest)
}

```

2. Configuración de Hilt

- **Clase Application:**
 - Crear una clase `RealEstateApplication` anotada con `@HiltAndroidApp` para inicializar Hilt.
 - Declararla en el `AndroidManifest.xml` con `android:name=".RealEstateApplication"`.
- **Módulo de Inyección:**
 - Crear un módulo Dagger (`AppModule`) anotado con `@Module` y `@InstallIn(SingletonComponent::class)` para proveer dependencias como `ApiService`, `PropertyDao`, `OkHttpClient`, `Retrofit`, y `AppDatabase`.
 - Usar `@Provides` y `@Singleton` para asegurar instancias únicas.
- **Anotaciones en clases:**
 - Anotar `MainActivity` con `@AndroidEntryPoint`.
 - Anotar `MainViewModel` con `@HiltViewModel` y usar `@Inject` constructor para inyectar `ApiService` y `PropertyDao`.

3. Configuración de Room

- **Base de Datos:**

- Crear una clase abstracta `AppDatabase` que extienda `RoomDatabase`, anotada con `@Database(entities = [Property::class], version = 1, exportSchema = false)`.
- Definir un método abstracto `propertyDao(): PropertyDao`.
- **DAO:**
 - Crear una interfaz `PropertyDao` con anotación `@Dao`.
 - Incluir métodos como `suspend fun insertAll(properties: List<Property>)` y `suspend fun getAll(): List<Property>` para operaciones asíncronas.
- **Entidad:**
 - Definir una data class `Property` anotada con `@Entity(tableName = "properties")`.
 - Usar `@PrimaryKey` para el campo `id` y `@SerializedName` para mapear campos del JSON.
- **Proveer en Hilt:**
 - En `AppModule`, proveer `AppDatabase` con `Room.databaseBuilder` y `PropertyDao` con `database.propertyDao()`.

4. Configuración de Retrofit y OkHttp

- **OkHttpClient:**
 - Configurar un `OkHttpClient` con un interceptor que añada el header `X-API-Key` (ej. `"realestatepro_secret_2025"`).
 - Proveerlo en `AppModule` con `@Provides` y `@Singleton`.
- **GSON:**
 - Configurar un `Gson` con `FieldNamingPolicy.IDENTITY` y `setLenient()` para un parseo flexible del JSON.
 - Proveerlo en `AppModule`.
- **Retrofit:**
 - Configurar un `Retrofit` con:
 - `baseUrl` (ej. `"http://192.168.0.20:8000/"`).
 - `client` (el `OkHttpClient` configurado).
 - `addConverterFactory(GsonConverterFactory.create(gson))`.
 - Proveerlo en `AppModule`.
- **Interfaz API:**
 - Crear una interfaz `ApiService` con métodos como `suspend fun getProperties(): List<Property>` anotados con `@GET("properties")`.
 - Proveerla en `AppModule` con `retrofit.create(ApiService::class.java)`.

5. Configuración de Red en AndroidManifest.xml

- **Permisos:**

- Añadir `<uses-permission android:name="android.permission.INTERNET" />` para permitir conexiones de red.
- **Tráfico claro:**
 - Incluir `android:usesCleartextTraffic="true"` en el elemento `<activity>` o `<application>` si la API usa HTTP (no HTTPS).
- **Configuración de seguridad:**
 - Crear un archivo `res/xml/network_security_config.xml` para permitir tráfico claro a dominios específicos (ej. 192.168.0.20).
 - Referenciarlo en el manifiesto con `android:networkSecurityConfig="@xml/network_security_config"`.

Ejemplo de `network_security_config.xml`:

xml

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <domain-config cleartextTrafficPermitted="true">
    <domain includeSubdomains="true">192.168.0.20</domain>
  </domain-config>
</network-security-config>
```

6. Configuración Adicional

- **Compose en Gradle:**
 - Habilitar Compose con `buildFeatures { compose = true }` en `build.gradle`.
- **Compatibilidad Java/Kotlin:**
 - Configurar `compileOptions` y `kotlinOptions` con `JavaVersion.VERSION_11` para compatibilidad.
- **Manifiesto:**
 - Asegurar que la actividad principal (`MainActivity`) tenga un `<intent-filter>` con `android.intent.action.MAIN` y `android.intent.category.LAUNCHER`.
 - Incluir configuraciones como `android:theme`, `android:label`, y `android:exported="true"`.

Consejos para Evitar Futuros Patatuses

- **Verificar Hilt:**
 - Siempre incluir y registrar la clase `Application` con `@HiltAndroidApp`.

- Asegurar que todas las dependencias (ej. `ApiService`, `PropertyDao`) estén proveídas en un módulo Dagger.
- **Probar Room:**
 - Validar que las entidades y el DAO estén bien definidos antes de compilar.
 - Usar KSP para procesar anotaciones de Room y Hilt.
- **Configurar red correctamente:**
 - No olvidar el permiso `INTERNET` ni el `networkSecurityConfig` para APIs locales.
 - Incluir el interceptor de OkHttp para headers como `X-API-Key`.
- **Manejo de errores:**
 - Implementar manejo robusto de excepciones en `ViewModel` (ej. `HttpException`, 403) y fallback a datos locales con Room.
- **Validar Gradle:**
 - Usar `libs.versions.toml` para centralizar versiones y evitar conflictos.
 - Incluir todas las dependencias necesarias para Compose, Hilt, Room, y Retrofit.

Con este resumen, tenemos una guía sólida para configurar un proyecto como este sin que nos vuelva a dar un patatús. Si quieres que profundice en alguna parte (ej. pruebas, optimización de Room, o más detalles de Retrofit), ¡avísame, mi brother! 